

Databáze

Co je databáze

- **databáze** = organizovaná kolekce dat
- slouží k ukládání, vyhledávání a správě dat
- data jsou uložena tak, aby se s nimi dalo efektivně pracovat
- databázi spravuje **databázový systém**

Příklady použití databází:

- evidence studentů
- e-shop
- bankovní systém
- sociální síť
- rezervační systém
- školní informační systém

DBMS

- **DBMS** = Database Management System
- česky: systém řízení báze dat
- software pro práci s databází
- umožňuje:
 - vytvářet databáze
 - ukládat data
 - číst data
 - upravovat data
 - mazat data
 - nastavovat oprávnění
 - zajišťovat integritu dat

Příklady DBMS:

- PostgreSQL
- MySQL
- Microsoft SQL Server
- Oracle Database
- SQLite
- MongoDB

Relační databáze

Co je relační databáze

- **relační databáze** ukládá data do tabulek
- vychází z relačního modelu
- relační model definoval Edgar F. Codd
- tabulka se odborně nazývá **relace**
- řádky tabulky představují záznamy
- sloupce tabulky představují vlastnosti záznamů
- struktura dat je předem daná schématem
- pro práci se používá jazyk **SQL**

Příklady relačních databází:

- PostgreSQL
- MySQL
- Microsoft SQL Server
- SQLite
- Oracle Database

Tabulka

- **tabulka** = základní struktura relační databáze
- obsahuje řádky a sloupce
- reprezentuje jeden typ dat

Příklad tabulky Student :

Id	Jmeno	Prijmeni	Vek
1	Jan	Novák	18
2	Eva	Svobodová	19
3	Petr	Dvořák	18

Řádek

- **řádek** = jeden konkrétní záznam v tabulce
- například jeden student

Příklad:

1, Jan, Novák, 18

Sloupec

- **sloupec** = vlastnost záznamu
- má název a datový typ

Příklady sloupců:

- Id
- Jmeno
- Prijmeni
- Vek
- DatumNarozeni

Schéma databáze

- **schéma** = předpis struktury databáze
- určuje:
 - tabulky
 - sloupce
 - datové typy
 - klíče
 - vazby
 - omezení

Příklad:

```
CREATE TABLE Student (  
    Id INT PRIMARY KEY,  
    Jmeno VARCHAR(50),  
    Prijmeni VARCHAR(50),  
    Vek INT  
);
```

SQL

Co je SQL

- **SQL** = Structured Query Language
- jazyk pro práci s relačními databázemi
- používá se pro:
 - vytváření tabulek
 - vkládání dat
 - čtení dat
 - úpravu dat
 - mazání dat

- správu práv

Základní SQL příkazy

SELECT

- slouží ke čtení dat

```
SELECT * FROM Student;
```

INSERT

- slouží ke vložení dat

```
INSERT INTO Student (Id, Jmeno, Prijmeni, Vek)  
VALUES (1, 'Jan', 'Novák', 18);
```

UPDATE

- slouží k úpravě dat

```
UPDATE Student  
SET Vek = 19  
WHERE Id = 1;
```

DELETE

- slouží ke smazání dat

```
DELETE FROM Student  
WHERE Id = 1;
```

Pozor na WHERE

- WHERE určuje, kterých řádků se příkaz týká
- bez WHERE se může změnit nebo smazat celá tabulka

Příklad nebezpečného příkazu:

```
DELETE FROM Student;
```

- smaže všechny studenty

NoSQL databáze

Co je NoSQL

- **NoSQL** = Not Only SQL
- databáze, které nejsou založené pouze na klasickém relačním modelu
- často nemají pevné schéma
- hodí se pro data, která se často mění
- často se používají pro velká a různorodá data
- mohou být velmi rychlé a dobře škálovatelné

Rozdíl SQL vs. NoSQL

SQL databáze	NoSQL databáze
data v tabulkách	data různými způsoby
pevné schéma	často bez pevného schématu
používá SQL	různé dotazovací způsoby
dobré pro vazby a transakce	dobré pro škálování a flexibilitu
časté JOINy	JOINy často omezené nebo chybí
PostgreSQL, MySQL	MongoDB, Redis, Neo4j

Typy NoSQL databází

Dokumentové databáze

- ukládají data jako dokumenty
- často ve formátu podobném JSON
- každý dokument může mít trochu jinou strukturu

Příklad dokumentu:

```
{
  "id": 1,
  "jmeno": "Jan",
  "prijmeni": "Novák",
  "predmety": ["Matematika", "Programování"]
}
```

Příklad systému:

- MongoDB

Použití:

- e-shopy

- CMS
- katalogy produktů
- aplikace s proměnlivou strukturou dat

Key-value databáze

- ukládají dvojice:
 - klíč
 - hodnota
- velmi rychlé vyhledávání podle klíče
- jednoduchý model

Příklad:

```
"user:1" → "Jan Novák"  
"session:abc123" → "data relace"
```

Příklad systému:

- Redis

Použití:

- cache
- session management
- rychlé ukládání jednoduchých dat

Grafové databáze

- ukládají data jako uzly a hrany
- hodí se pro data s mnoha vztahy

Příklad:

```
Jan --zná--> Eva  
Eva --zná--> Petr
```

Příklad systému:

- Neo4j

Použití:

- sociální sítě
- doporučovací systémy
- vztahy mezi objekty

- mapování propojení

Sloupcové databáze

- ukládají data po sloupcích
- hodí se pro velká množství dat
- často se používají v analytice

Příklad systému:

- Cassandra

Použití:

- big data
- analytické systémy
- rozsáhlé logy

Datové modelování

Konceptuální model

- řeší, **co** chceme v databázi evidovat
- neřeší konkrétní databázový systém
- často se používá **ER diagram**
- pracuje s entitami, atributy a vztahy

Entita

entita = objekt reálného světa, o kterém ukládáme informace

Příklady entit:

- Student
- Učitel
- Třída
- Auto
- Objednávka
- Produkt

Entitní typ a instance entity

Entitní typ

- obecná šablona entity

Příklad:

- Student

Instance entity

- konkrétní výskyt entity

Příklad:

- student Jan Novák

Atribut

- **atribut** = vlastnost entity

Příklad entity Student :

- jméno
- příjmení
- věk
- datum narození
- e-mail

Vztah

- **vztah** = propojení mezi entitami

Příklady:

- student patří do třídy
- objednávka obsahuje produkty
- učitel učí předmět

Logický model

- převádí konceptuální návrh do tabulek
- řeší:
 - tabulky
 - sloupce
 - primární klíče
 - cizí klíče
 - vazby mezi tabulkami

Příklad:

- entita Student se převede na tabulku Student
- atribut jméno se převede na sloupec Jmeno

Fyzický model

- konkrétní implementace v databázovém systému
- řeší:
 - přesné datové typy
 - indexy
 - omezení
 - fyzické uložení dat
 - výkon

Příklad:

```
Jmeno VARCHAR(50)
Vek INT
DatumNarozeni DATE
```

Klíče

Primární klíč

- **primární klíč** = jednoznačný identifikátor záznamu
- označuje se **PK**
- každý řádek má jinou hodnotu primárního klíče
- nesmí být `NULL`
- v tabulce bývá často sloupec `Id`

Příklad:

Id	Jmeno	Prijmeni
1	Jan	Novák
2	Eva	Svobodová

- `Id` je primární klíč

Cizí klíč

- **cizí klíč** = odkaz na primární klíč v jiné tabulce
- označuje se **FK**
- používá se pro propojení tabulek
- zajišťuje referenční integritu

Příklad:

Tabulka Trida :

Id	Nazev
1	4.A
2	4.B

Tabulka Student :

Id	Jmeno	TridaId
1	Jan	1
2	Eva	1
3	Petr	2

- Student.TridaId je cizí klíč
- odkazuje na Trida.Id

Kandidátní klíč

- sloupec nebo kombinace sloupců, které by mohly být primárním klíčem
- musí jednoznačně identifikovat řádek

Příklad:

- Id
- rodné číslo
- e-mail

Složený klíč

- klíč složený z více sloupců
- používá se například u vazebních tabulek

Příklad:

StudentId + PredmetId

Vazby mezi tabulkami

Vazba 1:1

- jednomu záznamu odpovídá právě jeden záznam v druhé tabulce
- používá se méně často

- někdy se dá sloučit do jedné tabulky

Příklad:

- člověk má jeden občanský průkaz
- občanský průkaz patří jednomu člověku

Vazba 1:N

- jednomu záznamu odpovídá více záznamů v druhé tabulce
- nejčastější typ vazby

Příklad:

- jedna třída má mnoho studentů
- jeden student patří do jedné třídy

Realizace:

- cizí klíč se dá na stranu N

Příklad:

```
Student.TridaId → Trida.Id
```

Vazba M:N

- jednomu záznamu odpovídá více záznamů v druhé tabulce
- zároveň jeden záznam z druhé tabulky odpovídá více záznamům v první tabulce
- relační databáze ji běžně řeší pomocí vazební tabulky

Příklad:

- student navštěvuje více předmětů
- jeden předmět má více studentů

Řešení:

- vytvoří se vazební tabulka

Příklad:

```
StudentPredmet(StudentId, PredmetId)
```

Tím vzniknou dvě vazby 1:N:

Student → StudentPredmet
Predmet → StudentPredmet

Speciální typy vazeb

Rekurzivní vazba

- entita má vztah sama k sobě
- tabulka odkazuje sama na sebe

Příklad:

- zaměstnanec má nadřízeného
- nadřízený je také zaměstnanec

Příklad:

```
Zamestnanec(Id, Jmeno, NadrizenyId)
```

- NadrizenyId odkazuje na Zamestnanec.Id

IS-A vazba

- vyjadřuje generalizaci a specializaci
- podobá se dědičnosti v objektovém programování
- jedna obecná entita má specializované podtypy

Příklad:

- Vozidlo
 - Auto
 - Motorka

Význam:

- Auto **je** vozidlo
- Motorka **je** vozidlo

Možnosti realizace:

- jedna společná tabulka
- více tabulek se společným ID
- tabulka pro předka a tabulky pro potomky

Integrita dat

Co je integrita dat

- **integrita dat** = správnost, konzistence a smysluplnost dat
- databáze se snaží zabránit nesprávným datům
- používají se:
 - datové typy
 - klíče
 - omezení
 - transakce

Doménová integrita

- hlídá, jaké hodnoty mohou být ve sloupci
- souvisí s datovým typem a omezeními

Příklady:

- věk nesmí být záporný
- e-mail nesmí být prázdný
- známka musí být v rozsahu 1 až 5
- datum narození musí být platné datum

Prostředky:

- datové typy
- NOT NULL
- CHECK
- DEFAULT
- UNIQUE

Příklad:

```
Vek INT CHECK (Vek >= 0)
```

Entitní integrita

- každý záznam musí být jednoznačně identifikovatelný
- zajišťuje ji hlavně primární klíč
- primární klíč nesmí být NULL
- primární klíč se nesmí opakovat

Referenční integrita

- hlídá vztahy mezi tabulkami

- zajišťuje ji cizí klíč
- nesmí existovat odkaz na neexistující záznam

Příklad:

- student nemůže patřit do třídy, která neexistuje

Chování při mazání návazných dat

RESTRICT

- zakáže smazání záznamu, pokud na něj někdo odkazuje

Příklad:

- nejde smazat třídu, pokud v ní jsou studenti

CASCADE

- smaže záznam i navázané záznamy

Příklad:

- smažu třídu
- smažou se i studenti v této třídě

Pozor:

- používá se opatrně
- může smazat hodně dat

SET NULL

- při smazání odkazovaného záznamu nastaví cizí klíč na `NULL`

Příklad:

- smažu třídu
- studentům se `TridaId` nastaví na `NULL`

Transakce

Co je transakce

- **transakce** = skupina databázových operací, které se berou jako jeden celek
- buď se provedou všechny
- nebo se neprovede žádná
- používají se pro zachování konzistence dat

Příklad:

- převod peněz mezi účty
- odečtení peněz z jednoho účtu
- přičtení peněz na druhý účet
- musí proběhnout obě operace, nebo žádná

ACID

Transakce by měly splňovat vlastnosti **ACID**.

Atomicita

- transakce se provede celá, nebo vůbec
- pokud nastane chyba, změny se vrátí zpět

Konzistence

- transakce převede databázi z jednoho platného stavu do jiného platného stavu
- nesmí porušit pravidla databáze

Izolace

- současně běžící transakce se nemají navzájem chybně ovlivňovat

Trvalost

- po potvrzení transakce se změny uloží natrvalo
- změny zůstanou i po pádu systému

COMMIT a ROLLBACK

COMMIT

- potvrdí transakci
- změny se uloží natrvalo

ROLLBACK

- zruší transakci
- vrátí databázi do původního stavu

Příklad:

```
BEGIN TRANSACTION;
```

```
UPDATE Ucet SET Zustatek = Zustatek - 1000 WHERE Id = 1;
```

```
UPDATE Ucet SET Zustatek = Zustatek + 1000 WHERE Id = 2;
```

```
COMMIT;
```

Pokud nastane chyba:

```
ROLLBACK;
```